User Authentication in Blazor Using ASP.NET Identity – Technical Documentation

1. Project Overview

This project demonstrates the implementation of user authentication in a Blazor application using ASP.NET Identity. The system provides secure login and logout functionality while ensuring proper authorization and session handling.

2. Objectives

- Implement secure user login and logout
- Validate authenticated users using ASP.NET Identity
- Protect authorized pages using the [Authorize] attribute
- Provide a clean and user-friendly login interface
- Maintain scalable and secure authentication architecture

3. Technologies Used

- Blazor
- ASP.NET Core / C#
- ASP.NET Identity
- Entity Framework Core
- SQL Server
- Visual Studio

4. Authentication Features

- User login functionality
- User logout functionality
- Session-based authentication
- Authorization checks
- Protected routes and components

5. Login User Interface

The login page provides fields for username and password. It uses ASP.NET Identity to validate user credentials securely before granting access to protected areas of the application.

6. Authentication Workflow

1. User navigates to the login page
2. User enters valid credentials
3. ASP.NET Identity validates the credentials
4. Authentication cookie/session is created
5. User is redirected to the authorized landing page
6. Unauthorized users are redirected back to the login page

7. Authorization Handling

Protected pages and components utilize the [Authorize] attribute. If an unauthenticated user attempts to access a protected resource, they are automatically redirected to the login page.

8. Logout Functionality

The logout process terminates the user session securely by clearing authentication cookies and redirecting the user back to the login page.

9. Database Integration

ASP.NET Identity stores user credentials, roles, and claims in a SQL Server database. Passwords are hashed and salted for security.

10. Security Considerations

- Password hashing and salting
- Secure authentication cookies
- Protection against unauthorized access
- Proper session termination on logout
- SQL injection prevention via Entity Framework

11. Future Enhancements

- Role-based authorization
- Two-factor authentication (2FA)
- Password reset and recovery

- Account lockout after multiple failed attempts
- External authentication providers (Google, Microsoft, etc.)

12. Use Cases

- Secure web applications
- Admin dashboards
- Internal business systems
- Enterprise Blazor applications

13. Conclusion

This project provides a solid foundation for implementing authentication in Blazor applications using ASP.NET Identity. It follows industry best practices for security, scalability, and maintainability.